

When Repair Absorbs Learning: Repair, Information Access, and Sparse-Observation Traffic Engineering in LEO Constellations

Linhui Wei, He Ma, Yumei Wang, *Member, IEEE*, and Guangteng Fan

Abstract—Learning-based traffic engineering (TE) systems increasingly pair a neural allocator with a repair layer that restores allocations to feasibility, and under sparse traffic observation — the norm in low-Earth-orbit satellite constellations, where only a subset of demands is metered — the learned component is additionally expected to impute the missing demand information. The two components are tacitly assumed to compose: learning refines the allocation and repair fixes whatever remains. We show that this assumption fails. In a controlled study of a sparse-input TE system on a 1,584-satellite Starlink constellation with population-driven traffic dynamics, we find that the repair layer *absorbs* the learning signal: repair behaves as a switch rather than a dial — a single repair sweep collapses the training gain from +28% to zero and further sweeps change nothing — so under the oracle-fed repair current systems inherit, a trained network is indistinguishable from an untrained one (9/25 paired wins, and a 95% interval on the paired gain that rules out any improvement beyond 0.6%). Learning pays off only in a repair-free regime, but does so across the entire observability range tested (13/15 paired wins, $\rho \in [0.02, 0.5]$), peaking at +28.1% at $\rho = 0.3$; no collapse boundary exists even at $\rho = 0.02$, where two of three seeds still improve. The learned system's remaining edge over zero-filling is stability — lower variance and better worst-case maximum link utilization — rather than median performance. A one-third-scale constellation instance reproduces the same mechanism. Finally, we show that this repair layer — inherited from the fully-observed setting it was designed for — silently consults the *ground-truth* traffic matrix to rank congested paths, information a sparsely-measuring controller does not have. Restricting repair to the observation it can actually see removes much of its advantage, which moves the operative question from whether learning helps to which component is allowed to see the traffic. These findings redraw the boundary of where learned TE adds value and yield concrete guidance for combining learning with feasibility repair.

Index Terms—Satellite networks, traffic engineering, sparse measurement, graph neural networks, constraint repair, mechanism analysis.

1 INTRODUCTION

LOW-EARTH-ORBIT (LEO) satellite constellations are rapidly scaling toward tens of thousands of satellites [1], [2], [3], carrying user traffic with pronounced spatial and temporal fluctuations over inter-satellite links (ISLs) of limited capacity [4], [5]. Traffic engineering (TE) — centrally optimizing how each demand is split across candidate paths — has become a key control primitive for these networks [6], [7], [8], [9]. A recent line of learning-based TE systems makes centralized control practical at scale: neural allocators replace online solvers and cut allocation time from minutes to milliseconds [10], [11], [12], [9]. Because a neural allocator can emit allocations that overload individual links, these systems almost universally append a *repair layer* — a few steps of a convex procedure such as ADMM [13], or a lightweight rebalancing heuristic, that restores the raw allocation to feasibility. Under sparse traffic observation, the learned component carries a second responsibility: since only a subset of demands is metered, it is also expected to *impute* the missing demand information, whether through

learned mask embeddings [10], synthetic augmentation [14], or explicit completion stages.

This second responsibility is unavoidable in LEO constellations. Network-wide measurement requires every satellite to meter, aggregate, and downlink fine-grained flow statistics through bandwidth-constrained telemetry channels, competing with user traffic and consuming scarce onboard resources; measurement gaps due to intermittent ground contacts are the norm rather than the exception. In practice, the controller observes a *sparse* traffic matrix — a fraction ρ of demands measured at each epoch — and must still allocate *all* traffic. Filling unobserved entries with zeros leaves no headroom on the links that unmeasured traffic actually traverses, inflating the realized maximum link utilization (MLU); two-stage complete-then-optimize pipelines propagate reconstruction errors into allocation decisions [14]. Learned imputation is therefore the standard answer: the allocator is trained end-to-end to act well despite missing inputs.

The two responsibilities — imputation and allocation — are trained into one network, while feasibility is enforced by the repair layer. This decomposition carries a tacit but load-bearing assumption: *the components compose*. The learned allocator is assumed to contribute whatever improvement it can learn from data, and the repair layer is assumed to add, on top of it, the correction of residual violations — two positive, additive terms. Every system that couples a neural

- L. Wei and G. Fan are with the National Innovation Institute of Defense Technology, Academy of Military Science, Beijing, China (e-mail: weilinhui998@163.com; fanguangteng@163.com).
- H. Ma and Y. Wang are with the School of Artificial Intelligence, Beijing University of Posts and Telecommunications, Beijing, China (e-mail: mahe@bupt.edu.cn; ymwang@bupt.edu.cn).
- Manuscript received XXX; revised XXX. (Corresponding author: Guangteng Fan.)

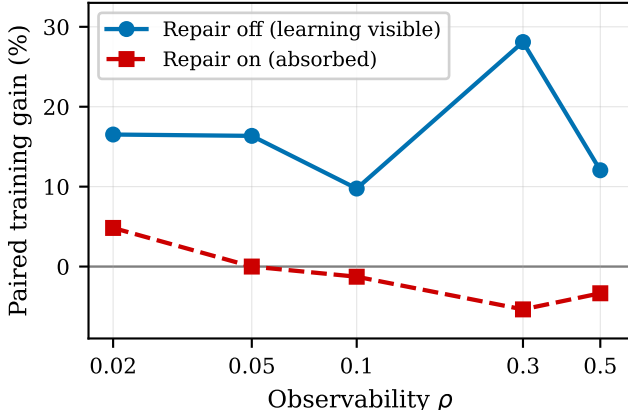


Fig. 1. The headline finding. With the repair layer enabled, training yields no measurable gain at any observability level. Disable repair, and training helps at *every* observability level tested — there is no collapse boundary, even at $\rho = 0.02$.

allocator with a repair layer [10], [9], [11] implicitly relies on this assumption when it reports end-to-end performance: the measured gain is attributed to the pipeline as a whole, and no attempt is made to separate what the learned part contributes from what the repair part contributes.

This paper shows that the assumption fails, and characterizes precisely how. Our vehicle is SiTE (Sparse-input Traffic Engineering), a representative sparse-input TE system that combines learnable mask embeddings, mask-aware gated message passing, temporal-spatial cross-attention, and a constraint-repair layer (section 4). We conduct a controlled mechanism study of SiTE on a 1,584-satellite Starlink constellation (Shell-1 configuration) driven by population-based traffic dynamics, holding the architecture fixed and sweeping observability ρ , the repair budget, training status, and random seeds. **(RQ1) Does learning contribute when repair is present?** No. The repair layer *absorbs* the learning signal, and it does so as a *switch* rather than a dial: a single repair sweep collapses the training gain from +28.1% to −3.3%, and two or five sweeps change nothing further. Once that oracle-fed repair is in place, a trained network is indistinguishable from an untrained one (9 of 25 paired snapshots won; sign test $p = 0.115$ in the untrained direction, and a 95% interval on the paired gain of [−5.5%, +0.6%] that rules out any improvement beyond 0.6%). **(RQ2) When does learning contribute without repair?** At *every* ρ tested: training wins 13 of 15 paired cells across $\rho \in [0.02, 0.5]$ (two-sided sign test $p = 0.018$), improving MLU by +28.1% at $\rho = 0.3$ and +9.8% at $\rho = 0.1$; even at $\rho = 0.02$ two of three seeds improve. No collapse boundary exists — the mask embeddings regularize allocations even at 2% observability. **(RQ3) Does the mechanism depend on constellation scale?** A one-third-scale instance of the same constellation architecture reproduces the absorption phenomenon (section 5). **(RQ4) What does repair need to know?** More than the policy does. To rank congested paths, the repair step needs demand volumes, and the implementation inherited from the fully-observed setting takes them from the *ground-truth* matrix — an information advantage over a policy that sees only a frac-

tion ρ of it, and one that no sparsely-measuring controller could reproduce. Restricting repair to the observation it can actually see costs it a substantial part of its advantage, which reopens the space the learned component was supposed to occupy. Together, the results imply that the learned component and the repair component are *functional substitutes*, not complements — and that the substitution is underwritten by an information asymmetry that the literature’s evaluation protocol leaves invisible.

The findings redraw the boundary of where learned TE adds value, and they carry concrete guidance. *For evaluation*, a learned TE system paired with strong repair must be benchmarked in the repair-free regime or against repair-only baselines; end-to-end numbers conflate the two components and can certify a trained model whose weights contribute nothing. *For design*, if repair is kept for safety, the learning budget should target what repair provably does not provide — we show that the one robust residual advantage of the learned system is *stability*: it wins 17/25 paired snapshots against zero-filling, with markedly lower variance and a worst-case MLU up to 38% better at extreme sparsity — rather than median performance. *For deployment*, the absence of a collapse boundary means training remains worth its cost even at 2% observability; and the operative choice is not whether to pay for repair but which matrix to hand it — the variant that erases the learning signal is precisely the variant a sparsely-measuring deployment cannot run.

In summary, this paper makes the following contributions:

- We identify the *repair–learning composition assumption* tacitly made by learned TE systems that couple a neural allocator with a constraint-repair layer, and design a controlled protocol — varying training status, repair budget, observability, and seeds in isolation — to test it (sections 3 and 5).
- We show the assumption fails via a mechanism we call *repair absorption*: repair acts as a switch — one step collapses the entire learned differential, and further steps add nothing — while in absolute terms that same repair is what delivers most of the performance; without repair, learning is effective across the entire observability range tested, with no collapse boundary even at $\rho = 0.02$ (section 5).
- We characterize what learning *does* still provide — a stability advantage in variance and worst-case MLU — and translate the findings into guidance for evaluating, designing, and deploying learned TE under sparse observations, validated across constellation scales (sections 5 and 6).
- We show that the inherited repair layer owes much of its strength to reading the ground-truth traffic matrix — a dependence that is unremarkable under full observability but unavailable, and to our knowledge never isolated, under sparse measurement. Making the matrix repair reads an explicit factor separates the repair *algorithm* from the repair *information* (section 5.9).

2 RELATED WORK

2.1 ML-Driven Traffic Engineering for WANs

Traffic engineering has long been formulated as a constrained optimization problem [15], [16] solved by linear programming (LP), whose runtime scales poorly with network size. SMORE [17] decouples robust oblivious path selection from online rate adaptation, while NCFlow [18] and POP [19] accelerate LP by problem decomposition, trading allocation quality for speed. A recent line of work replaces online solvers with learned models [20]: DOTE [11] directly maps historical traffic matrices to split ratios with stochastic optimization; Teal [10] combines a flow-centric GNN, multi-agent RL, and ADMM fine-tuning to achieve near-optimal allocations with orders-of-magnitude speedups; FIGRET [21] enhances robustness against traffic uncertainty at per-flow granularity. To survive topology changes without retraining, HARP [12] enforces invariances to node permutation and tunnel reordering, transferring across topologies unseen in training. More recently, large language models have been explored as general-purpose traffic planners [22]. However, all of these systems assume the *complete* traffic matrix is available at decision time — an assumption that rarely holds in large-scale satellite networks, where network-wide measurement is prohibitively expensive.

2.2 TE and Routing for Satellite Networks

LEO mega-constellations pose unique challenges for TE: topologies change every few tens of milliseconds as satellites move, and control decisions must be produced within stringent latency budgets [1], [2], [4]. Classical approaches absorb mobility through topology abstractions — the virtual node model binds logical nodes to earth-fixed footprints so that upper-layer routing observes a quasi-static grid [23]. Optimization-based satellite TE has explored segment routing [24], distributed clustering that optimizes elephant flows separately [25], and supervised learning for distributed split-ratio prediction [26]. SaTE [9] is the state-of-the-art learning-based TE for satellite constellations: it formulates a heterogeneous graph over satellites, demands, paths, and links, prunes zero-traffic relations to make training tractable on a single GPU, and selects representative topologies via Graph2Vec and DPP sampling so that one trained model generalizes across topology snapshots with millisecond inference. In parallel, FNC [27], [28] targets dynamic traffic allocation for earth-observation constellations, replacing RL with end-to-end imitation learning and substituting iterative constraint solvers with normalization-based feasibility layers. Notably, FNC’s imitation learning requires expert demonstrations, and the TE expert is an optimal allocation whose computation needs the complete demand matrix of each episode — a paradigm that fundamentally requires complete post-hoc observability, which is unavailable under sparse measurement. Both SaTE and FNC, like their WAN counterparts, assume the traffic matrix is fully observable at decision time.

2.3 TE with Incomplete Traffic Observations

Network-wide traffic measurement is expensive: collecting a complete traffic matrix requires instrumenting all ingress

TABLE 1
Comparison with the closest prior systems.

	Teal [10]	SaTE [9]	TEST [14]	FNC [27]	Ours
Satellite scenario	×	✓	×	✓	✓
Dynamic topology	×	✓	×	✓	✓
Sparse observation	×	×	✓	×	✓
Temporal modeling	×	×	✓	×	✓
Mask-aware model	×	×	×	×	✓
Demand-set drift	×	partial	×	×	✓

nodes at fine time granularity, which is particularly onerous on resource-constrained satellite platforms. A classical workaround is a two-stage pipeline — first estimate or complete the traffic matrix via network tomography [29], [30], matrix completion, or compressive sensing [31], then optimize routing on the estimate — but reconstruction errors propagate to and degrade the downstream allocation. TEST [14] is, to our knowledge, the first end-to-end approach that learns routing policies directly from sparsely measured traffic matrices, integrating a Transformer over historical sparse sequences with a GCN over the topology; it fills unobserved entries with zeros, compensates with synthesized training augmentation, and targets terrestrial networks with static topologies. In the graph learning literature, message passing under missing node features has been addressed via teacher–student distillation [32], but such techniques have not been applied to network TE. No prior work, to our knowledge, performs TE from sparse traffic observations in satellite networks, where the difficulty is compounded by time-varying topologies and drifting demand sets.

2.4 Positioning

Our work sits at the intersection of these three lines (table 1). Unlike SaTE and FNC, which assume fully observable demands, we perform TE from sparse observations (10%–70% observability). Unlike TEST, which fills unobserved entries with zeros and targets static terrestrial networks, we replace missing entries with learnable mask embeddings, gate their message passing, and handle LEO dynamics — time-varying topologies and drifting demand-pair sets — via demand pruning with size-invariant model weights, enabling train-once, zero-retraining deployment.

3 MODEL AND PROBLEM FORMULATION

3.1 Network and Traffic Model

We model the constellation at control epoch t as a directed graph $G^t = (V, E^t)$, where V is the set of N satellites and E^t the set of inter-satellite links (ISLs), each link e with capacity c_e . Time is slotted into control epochs; within an epoch the topology snapshot is fixed, while across epochs both E^t and the traffic change as satellites move.

Traffic is described by a demand matrix $D^t \in \mathbb{R}^{N \times N}$, where d_i denotes the volume of the i -th source–destination pair. Since satellite traffic is geographically concentrated, we maintain an *active demand set* $\mathcal{K}^t$ — the pairs with nonzero demand in recent history (section 4) — and only these are modeled. Each demand $i \in \mathcal{K}^t$ is served by κ pre-selected

TABLE 2
Main Notations.

Symbol	Definition
$G^t = (V, E^t)$	constellation graph at epoch t ; c_e link capacity
D^t, d_i	demand matrix; volume of demand i
$\mathcal{K}^t$	active demand set at epoch t
κ, P_i	candidate paths per demand; path set of demand i
$r_i, f_{i,k}$	split ratios of demand i ; flow on path $p_{i,k}$
M, ρ	binary observation mask; observability ratio
O^t, H^t	sparse observation $D^t \odot M$; last- L history
ϕ	learnable mask embedding for unmeasured demands
$A, \tilde{A}$	bipartite adjacency; its mask-gated variant
H_S, H_T, H_F	spatial, temporal, and fused embeddings
π_θ, R	TE policy network; ground-truth reward

candidate paths $P_i = \{p_{i,1}, \dots, p_{i,\kappa}\}$ ($\kappa = 4$ edge-disjoint shortest paths in our implementation). The TE decision for demand i is a split-ratio vector $r_i \in \mathbb{R}^\kappa$ with $r_{i,k} \geq 0$ and $\sum_k r_{i,k} \leq 1$, allocating flow $f_{i,k} = r_{i,k} \cdot d_i$ to path $p_{i,k}$; the residual $1 - \sum_k r_{i,k}$ represents intentionally unserved traffic. Main notations are summarized in table 2.

3.2 TE with Complete Observation

With the complete demand matrix available, TE is the classic path-based allocation problem, for which two objectives dominate the literature. *Load balancing* minimizes the maximum link utilization (MLU) — the standard objective in satellite TE, where hot ISLs over populated regions dictate quality of service [9], [24], [25]:

$$\min_{\{r_{i,k}\}} \theta \quad (1)$$

$$\text{s.t.} \quad \frac{1}{c_{e(i,k): e \in p_{i,k}}} \sum r_{i,k} d_i \leq \theta, \quad \forall e \in E^t, \quad (2)$$

$$\sum_{k=1}^{\kappa} r_{i,k} = 1, \quad \forall i \in \mathcal{K}^t, \quad (3)$$

$$r_{i,k} \geq 0, \quad \forall i, k, \quad (4)$$

where (3) enforces flow conservation: every demand must be fully served, so θ cannot be lowered by simply withholding traffic. *Throughput maximization* instead maximizes the total satisfied demand, permitting demands to be served only partially when capacity is scarce (total-flow TE [10], [18]):

$$\max_{\{f_{i,k}\}} \sum_{i \in \mathcal{K}^t} \sum_{k=1}^{\kappa} f_{i,k} \quad (5)$$

$$\text{s.t.} \quad \sum_{k=1}^{\kappa} f_{i,k} \leq d_i, \quad \forall i \in \mathcal{K}^t, \quad (6)$$

$$\sum_{(i,k): e \in p_{i,k}} f_{i,k} \leq c_e, \quad \forall e \in E^t, \quad (7)$$

$$f_{i,k} \geq 0, \quad \forall i, k. \quad (8)$$

Both problems are linear programs solvable to optimality, but at constellation scale their solve time far exceeds the control epoch [9], and — more fundamentally for this paper — the link-load terms in (2) and (7), as well as the per-demand cap (6), all require knowing d_i for *every* active demand. We treat MLU minimization as the primary objective

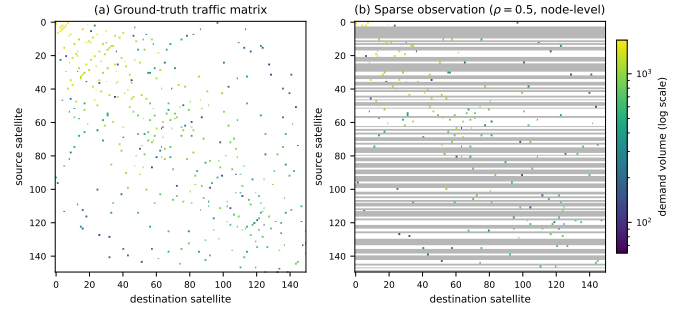


Fig. 2. A ground-truth traffic matrix from the Starlink 22×72 trace (150 busiest satellites shown) and its node-level sparse observation: white cells carry no traffic, gray rows are unmeasured source satellites, and colored cells are observed demands.

throughout this paper, following the satellite TE literature, and report throughput maximization as a secondary instantiation to show that our design is objective-agnostic.

3.3 TE with Sparse Observation

3.3.1 Observation Model

At each epoch only a subset of demands is measured. A binary mask $M \in \{0, 1\}^{|\mathcal{K}^t|}$ marks demand i as measured ($M_i = 1$) or not ($M_i = 0$), and the controller receives the sparse observation

$$O^t = D^t \odot M, \quad \text{together with } M \text{ itself}, \quad (9)$$

where $\odot$ is the element-wise product. Figure 2 contrasts a ground-truth traffic matrix from our Starlink trace with its node-level sparse observation. We consider two measurement granularities: *flow-level*, where a fraction ρ of demand pairs is sampled, and *node-level*, where a fraction ρ of satellites meter all their outgoing demands — the latter matching deployments where measurement capability resides on a subset of satellites. The controller additionally retains the last L sparse observations $H^t = (O^{t-L+1}, \dots, O^t)$.

3.3.2 Problem Statement

A key observation makes decision-making under unknown demands physically executable: the decision variables are split ratios, not absolute volumes. Once installed in the forwarding plane, r_i is applied multiplicatively to whatever traffic actually arrives, so a decision for demand i requires no knowledge of d_i . Sparse-observation TE then seeks a policy π that maps the observable state to split ratios for *all* active demands,

$$\{r_i\}_{i \in \mathcal{K}^t} = \pi(H^t, M, G^t), \quad (10)$$

maximizing the *ground-truth* objective. Let $f_{i,k}(\pi) = \tilde{r}_{i,k} \cdot d_i$ denote the flow actually delivered on path $p_{i,k}$, where $\{\tilde{r}_i\}$ are the policy's ratios after an optional repair step (section 4.6): ADMM refinement with rounding under the throughput objective, and conservation-preserving rebalancing under MLU, where the ratios are already conservation-feasible by (3) but may still overload individual links. Formally, for MLU minimization

$$\min_{\pi} \mathbb{E}_t \left[\max_{e \in E^t} \frac{1}{c_{e(i,k): e \in p_{i,k}}} \sum f_{i,k}(\pi) \right], \quad (11)$$

and, for the throughput variant,

$$\max_{\pi} \mathbb{E}_t \left[\sum_{i \in \mathcal{K}^t} \sum_{k=1}^{\kappa} f_{i,k}(\pi) \right], \quad (12)$$

where the expectations are over traffic and topology dynamics. Note that the link loads in (11) are evaluated with the true demands d_i : an unmeasured demand still consumes real capacity even though the policy could not see it, so misjudging it inflates the realized MLU. The problem is a partially observable sequential decision problem: unlike (1)–(8), where the same D^t appears in both the decision and the constraints, here the decision variables and the evaluation live in different information sets — the policy acts on (O, M) only, while feasibility and performance are settled by the hidden D . Intuitively, the policy is graded against an exam it never gets to read in full.

3.4 Solution Paradigm

Three solution families exist for (11) and (12). *Solve-with-zeros* plugs O directly into the LP (1)–(8), implicitly assuming unmeasured demands are zero; the resulting policy leaves no headroom on the links that unmeasured traffic actually traverses, so the realized MLU can be far above the optimum (and, under the throughput objective, unmeasured demands are structurally starved). *Two-stage* approaches first estimate $\hat{D}$ from (H, M) — via interpolation, tensor completion, or compressive sensing [29], [30], [31] — then solve the LP on $\hat{D}$; estimation is optimized for reconstruction error, which is misaligned with allocation quality, and errors propagate un-damped into the allocation. We instead adopt *end-to-end learning*: parameterize π_{θ} as a neural network and optimize θ directly against the TE objective,

$$\max_{\theta} \mathbb{E} [R(\pi_{\theta}(H, M, G), D)], \quad (13)$$

where R is the TE objective evaluated on the true D — negative realized MLU for (11), or total satisfied demand for (12). Since R is non-differentiable through the capacity-constrained network and the true D is available only as a training-time signal, we optimize (13) with multi-agent reinforcement learning — each demand acts as an agent sharing θ , receiving a COMA-style marginal-contribution reward (section 4) — followed by an optional lightweight repair step at inference that relieves overloaded links (section 4.6). This paradigm makes the observation model (9) a property of the *input*, and the optimization target (12) a property of the *training signal*, cleanly separating what the policy can see from what it is optimized for.

4 SiTE DESIGN

4.1 Overview

SiTE solves the sparse-observation TE problem of section 3.3 in four stages, as illustrated in fig. 3. Given the sparse observation (O^t, M) and history H^t , (i) unmeasured demands are represented by a learnable mask embedding (section 4.2); (ii) a mask-aware gated GNN propagates information between candidate paths and links without letting placeholders contaminate link states (section 4.3); (iii) a Transformer encodes the observation history and fuses it with spatial embeddings

Algorithm 1 SiTE inference at control epoch t

Require: sparse history H^t , mask M , topology G^t , weights θ^*

Ensure: split ratios $\{r_i\}_{i \in \mathcal{K}^t}$

- 1: $\mathcal{K}^t \leftarrow$ active demand set from recent history $\triangleright$ pruning, section 4.5
- 2: rebuild bipartite graph and gated adjacency $\tilde{A}$ from $\mathcal{K}^t, M$ $\triangleright$ eq. (15)
- 3: $x_i \leftarrow M_i d_i + (1 - M_i) \phi$ for all i $\triangleright$ mask embedding, eq. (14)
- 4: $H_S \leftarrow \text{GATEDFLOWGNN}(x, \tilde{A})$ $\triangleright$ spatial embeddings
- 5: $H_T \leftarrow \text{TEMPORALENCODER}(H^t)$ $\triangleright$ temporal embeddings
- 6: $H_F \leftarrow \text{CROSSATTENTION}(H_T, H_S)$ $\triangleright$ eq. (16)
- 7: $\{r_i\} \leftarrow \text{POLICYHEAD}([H_S \parallel H_F])$
- 8: $\{r_i\} \leftarrow \text{REPAIR}(\{r_i\}, G^t, \hat{D}^t)$ $\triangleright$ section 4.6
- 9: **return** $\{r_i\}$

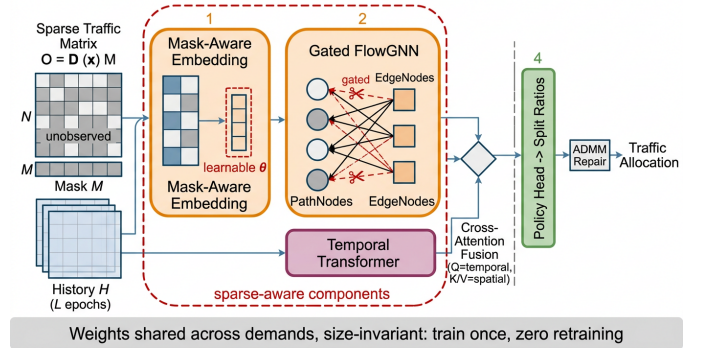


Fig. 3. SiTE overview: a sparse observation is embedded with mask awareness, processed by a gated flow-centric GNN in parallel with a temporal encoder, fused via per-demand cross-attention, and mapped to split ratios refined by the constraint-repair step.

via per-demand cross-attention (section 4.4); (iv) a shared policy head outputs split ratios per demand, refined by the constraint-repair step (section 4.6). All learnable weights are shared across demands and independent of the demand-set size, which section 4.5 exploits for zero-retraining deployment under constellation dynamics. Algorithm 1 summarizes one inference epoch.

4.2 Mask-Aware Demand Representation

A sparse observation is fundamentally ambiguous: a zero entry may mean “no traffic” or “not measured”. Zero-filling — the de-facto treatment in prior sparse TE [14] — collapses the two cases and systematically biases the allocator toward starving unmeasured demands. SiTE instead assigns each unmeasured demand a *learnable mask embedding* $\phi \in \mathbb{R}^{\kappa}$ (one scalar per candidate path):

$$x_i = M_i \cdot d_i + (1 - M_i) \cdot \phi, \quad (14)$$

so the model learns a data-driven prior for missing traffic during end-to-end training, and — critically — the input itself distinguishes “not measured” from “zero”. Learnable imputation has proven effective for incomplete graph features [32]; we bring it to TE, where the imputed value

directly steers resource allocation. The embedding is trained jointly with the policy under the TE objective (13) rather than a reconstruction loss, so ϕ converges to values that maximize allocation quality, not reconstruction accuracy.

4.3 Gated Message Passing

SiTE inherits Teal’s flow-centric graph [10]: candidate paths and physical links form a bipartite structure where a PathNode connects to the EdgeNodes it traverses, and message passing alternates between them with GCN-style normalized aggregation [33]. Under sparse observation, however, symmetric message passing is harmful: placeholder values on unmeasured PathNodes would propagate into EdgeNode embeddings and corrupt the network-wide estimate of link utilization — a phenomenon consistent with the feature–structure interference observed in incomplete graphs [32]. We therefore gate the propagation *asymmetrically*. Let A be the normalized bipartite adjacency; we zero out entries that carry path-to-edge messages from unmeasured paths while keeping all edge-to-path entries:

$$\tilde{A}[e, p] = M_p \cdot A[e, p], \quad \tilde{A}[p, e] = A[p, e], \quad (15)$$

where M_p is the demand-level mask lifted to path p . Unmeasured demands thus remain *listeners*: they sense link states — which are shaped by measured traffic — but inject no fabricated volume into them. The gate is a fixed binary function of the mask with no extra parameters, adding zero inference overhead. The asymmetric gate yields a formal isolation property:

Proposition 1 (Placeholder isolation). Under the gated propagation (15), the embeddings of all EdgeNodes and of all PathNodes belonging to measured demands are independent of the mask embedding ϕ ; ϕ influences only the allocations of unmeasured demands.

Proof: Consider first the spatial stream, by induction on GNN layers. At layer 0, only unmeasured PathNodes carry ϕ . At layer l , an EdgeNode aggregates path-to-edge messages with weights $\tilde{A}[e, p] = M_p A[e, p]$, which vanish for every unmeasured path; hence EdgeNode embeddings depend only on measured-path inputs and their own history, none of which contains ϕ . A measured PathNode aggregates EdgeNode embeddings (ϕ -free by the above) and its own ϕ -free input; the per-demand layers mix only paths of the same demand, which share the same mask value. The temporal and fusion streams preserve the property: the temporal encoder acts per path on the raw observation history H^t , which contains no ϕ by construction, and the cross-attention of (16) operates strictly within each demand, so the fused features of a measured demand combine only its own ϕ -free temporal and spatial embeddings. Thus ϕ never enters any ϕ -free branch, and only unmeasured PathNodes — and consequently their own split ratios — depend on ϕ . $\square$

In other words, learning a poor prior ϕ can at worst misallocate the unmeasured demands themselves, but can never corrupt the allocations of measured traffic — a robustness guarantee that zero-filling with ungated propagation does not provide.

4.4 Temporal-Spatial Fusion

Satellite traffic exhibits strong short-term temporal correlation; the recent history of sparse observations therefore carries recoverable information about currently missing entries. SiTE encodes, for each candidate path, its observation sequence over the last L epochs with a lightweight Transformer [34]: scalar volumes are log-compressed — which we find essential for training stability — and projected to d_{model} dimensions, summed with learnable positional embeddings, and passed through a Transformer encoder; the representation at the latest step forms the temporal embedding H_T . Rather than concatenating temporal and spatial features naively, we fuse them with *per-demand cross-attention*: within each demand, temporal embeddings of its κ candidate paths act as queries attending to the κ spatial (GNN) embeddings H_S as keys and values,

$$H_F = \text{softmax}\left(\frac{W_q H_T (W_k H_S)^\top}{\sqrt{d}}\right) W_v H_S, \quad (16)$$

and the fused features $[H_S \parallel H_F]$ feed the policy head. Intuitively, the temporal stream decides *which* spatial evidence to trust for each path: when the current entry is missing, attention shifts toward paths and links whose states corroborate the historical pattern. The full pipeline infers in under 0.4 s per snapshot on the 1,584-satellite constellation (section 5), well within the 5-minute control-loop period.

4.5 Scaling to Dynamic Constellations

4.5.1 Demand Pruning

Modeling all $N(N-1)$ pairs of a 1,584-satellite constellation yields 2.5M demands and tens of millions of path variables — infeasible for online control. Satellite traffic, however, is geographically concentrated: only 0.25% of pairs ever carry traffic in our dataset. SiTE instantiates PathNodes only for the demand-pair union observed in historical traffic matrices, shrinking the graph by $\sim 400\times$ with no loss of served traffic, in the same spirit as SaTE’s traffic pruning [9].

4.5.2 Zero-Retraining under Drift

Pruned demand sets are not static: as satellites move, the active pair set drifts (we measure $\sim 10\%$ turnover across our trace, section 5). Rather than retraining, SiTE exploits a structural property of its architecture: every learnable component — GNN layers, mask embedding, temporal encoder, fusion and policy heads — is shared across demands and independent of the demand-set size. The demand set, candidate paths, and the bipartite graph are runtime *inputs*, not parameters. At each control epoch the controller rebuilds the graph from the current active set — a millisecond-scale index operation on +Grid constellations, where candidate paths depend only on the orbital displacement between endpoints and reduce to $O(N)$ path templates — and reuses the trained weights as-is. Demands outside the modeled set — necessarily sporadic mice flows — fall back to shortest-path routing until the next set refresh. This reuse is justified by the following structural property:

Proposition 2 (Size invariance and permutation equivariance). The parameter set of SiTE is independent of $|\mathcal{K}^t|$,

N , and $|E^t|$, and the mapping from inputs to split ratios is equivariant to any permutation of the demand indices.

Proof: Every learnable component is a per-node or per-demand operator with shared weights: the mask embedding ϕ and temporal encoder act on each path token, the GNN layers combine node-wise linear maps with sparse aggregation, the fusion and policy heads act within each demand. None of their parameter shapes involves $|\mathcal{K}^t|$, N , or $|E^t|$. Permuting demand indices permutes the rows of every input and of the bipartite adjacency consistently; since shared node-wise operators commute with permutations and sparse aggregation is index-based, all intermediate embeddings — and hence the outputs — are permuted identically. $\square$

4.6 Training and Constraint Repair

SiTE trains with the multi-agent RL scheme of Teal [10]: each demand is an agent sharing the policy network. The policy outputs a Gaussian mean per path, and actions are sampled during training. To assign credit, each agent receives a COMA-style [35] counterfactual reward that estimates its marginal contribution to the global objective (13): for demand i ,

$$A_i = \mathbb{E}_{\tilde{r}_i \sim \pi'} [R(r_{-i}, r_i) - R(r_{-i}, \tilde{r}_i)], \quad (17)$$

where r_{-i} denotes the allocations of all other demands and $\tilde{r}_i$ is a counterfactual action resampled from a reference distribution π' ; the policy gradient weights each agent's log probability by A_i . Algorithm 2 summarizes offline training. Two points deserve emphasis. First, the reward is computed against *ground-truth* demands during offline training — sparse observation constrains the policy input, never the training signal — which is exactly what a simulator or historical trace provides. Second, we deliberately retain reward-driven RL rather than imitation learning [27]: imitation learning requires expert allocations solved from *fully realized* demands of each episode, a supervision signal that is unavailable when even post-hoc observation is sparse; RL needs only the reward, which the network itself provides.

Constraint repair. Under the throughput objective, allocations are refined by two parallelizable ADMM [13] iterations and a rounding pass that together repair residual violations of the capacity constraint (7). Under the MLU objective — our primary setting — the situation differs, and the difference turns out to govern the rest of this paper. The softmax ratios already satisfy flow conservation (3) by construction, so there is no infeasibility to project away; what they can still do is overload individual links. The repair step is therefore not a projection but a *conservation-preserving rebalancing*: for each demand it repeatedly shifts a fraction of the flow from its most congested candidate path — ranked by the bottleneck utilization of the links that path traverses — onto its least congested one, leaving per-demand totals untouched so that MLU cannot be lowered by under-serving traffic. One unit of repair budget corresponds to ten such sweeps.

What repair needs to know. Ranking paths by bottleneck utilization requires link loads, and link loads require demand volumes. In the fully-observed WAN setting this repair mechanism was designed for, those volumes are

Algorithm 2 Offline training of SiTE

Require: historical trace $\{(D^t, G^t)\}$, mask M , learning rate η

Ensure: trained weights θ^* (incl. mask embedding ϕ)

- 1: build demand set $\mathcal{K}$ and graph from training-slice TMs
- 2: **for** each epoch **do**
- 3: **for** each snapshot t in the training slice **do**
- 4: form sparse input (H^t, M) ; sample actions $r \sim \pi_\theta(\cdot | H^t, M, G^t)$
- 5: compute counterfactual rewards $\{A_i\}$ via (17)
- 6: **using the true** D^t
- 7: $\theta \leftarrow \theta + \eta \sum_i A_i \nabla_{\theta} \log \pi_\theta(r_i)$
- 8: **end for**
- 9: **return** θ^*

available at decision time, so consulting them is unremarkable. Under sparse observation they are not: a controller that meters only a fraction ρ of demands cannot compute the true utilization. We therefore promote *which matrix repair reads* to an explicit factor of the study, with three settings — ORACLE, which reads the ground-truth matrix and reproduces the inherited implementation; OBS-ZERO, which reads the sparse observation with unobserved demands zero-filled; and OBS-NBR, which fills them with the neighbor estimate of section 4.2. Only the latter two are deployable under sparse measurement. Section 5 shows that this distinction — invisible in the fully-observed setting, and to our knowledge never isolated before — decides whether the learned component contributes at all.

4.7 Complexity Analysis

Let $D = |\mathcal{K}^t|$ be the number of active demands, κ the candidate paths per demand, E the number of ISLs, and $\bar{\ell}$ the average path length. The gated FlowGNN performs message passing over the bipartite graph whose edge count is $O(D\kappa\bar{\ell})$; with Λ layers its cost is $O(\Lambda D\kappa\bar{\ell}d)$ for hidden width d . The temporal encoder applies a Transformer of length L independently to each of the $D\kappa$ path tokens, costing $O(D\kappa L^2 d)$, and the per-demand cross-attention over κ paths adds $O(D\kappa^2 d)$. Since κ , L , $\bar{\ell}$, and d are small constants fixed by design, the end-to-end inference cost is *linear* in the number of active demands, $O(D)$, and all operations are batched across demands on the GPU. Crucially, the parameter count is independent of D , N , and E : the same weights apply to any demand set, which is what enables zero-retraining reuse (section 4.5). In our configuration the entire model has merely 4,468 trainable parameters — small enough to be replicated on resource-constrained satellite platforms. Demand pruning reduces D from $N(N-1)$ to the observed support — roughly $400\times$ on our constellation — turning an otherwise intractable problem into one solved well within the control period (section 5.8).

5 EVALUATION

5.1 Setup and Protocol

Constellation. Our main testbed is a Starlink Shell-1 Walker constellation [3] interconnected in a +Grid pattern (table 3);

TABLE 3
Simulation parameters of the Starlink constellation.

Parameter	Value
Height of LEO satellites	550 km
Constellation configuration	Walker
Total number of satellites	1,584
Number of orbital planes	72
Satellites per plane	22
Inclination of orbits	55°
Phase factor	3
Inter-satellite links	6,336

TABLE 4
Statistics of the population-based traffic trace (Mbps).

Metric	Value
Snapshots (5-min interval)	101
Nonzero demand, max	1,800
Nonzero demand, mean $\pm$ std	159.5 $\pm$ 242.3
Total demand per snapshot	890,242
Active pairs per snapshot	5,583
Active-pair union (trace)	6,334

orbital dynamics are derived from the Satellite Tool Kit (STK). With 1,584 satellites, an average path length of 23.51 hops, and a diameter of 47, capacity conflicts between demands are far more frequent than in the terrestrial WANS used in prior TE studies [10]. For the cross-scale check (section 5.6) we use a one-third-scale instance of the same architecture (24 planes $\times$ 22 satellites, 528 nodes).

Traffic. Traffic matrices are generated by random city sampling [36]: cities are drawn with probability proportional to their population, each sampled city pair is attached to its nearest satellites, and per-pair flows are aggregated into satellite-level demands (table 4). We use 101 consecutive snapshots (one every 5 minutes), with snapshots 0–79 for training, 80–89 for validation, and 90–100 for testing — a strict temporal split with no leakage. ISL capacity is calibrated so that the fully-observed optimum leaves moderate congestion, keeping TE decisions non-trivial. Demand pruning (section 4.5) restricts the modeled demand set to the $\sim 6.3K$ pairs that ever carry traffic, out of 2.5M node pairs.

Protocol. Our metric is the *realized MLU* evaluated against ground-truth demands (lower is better); flow conservation (3) is enforced by construction, so MLU cannot be gamed by under-serving traffic. The study varies five factors independently: observability $\rho \in \{0.02, 0.05, 0.1, 0.3, 0.5\}$; the repair budget $\in \{0, 1, 2, 5\}$ sweeps (2 is the deployed default); the matrix the repair step is allowed to read (section 4.6) — the ground-truth one (ORACLE, the inherited default) or the sparse observation (OBS-ZERO, OBS-NBR); training status (trained vs. an *untrained* control sharing the identical architecture, mask embeddings, and repair layer, but with weights frozen at random initialization); and five random seeds. Unless stated otherwise, “repair on” means the ORACLE setting, so that sections 5.2 to 5.4 reproduce the configuration a Teal-style system would inherit; section 5.9 then removes that assumption. Because seed-to-seed variation turned out to be comparable to between-configuration differences, all headline comparisons are *paired* on (ρ , seed) and assessed with two-sided sign

TABLE 5
Model and training hyperparameters (defaults held fixed across all ρ and seeds).

Parameter	Value
FlowGNN layers Λ	6
Candidate paths κ	4
Temporal d_{model} / heads / layers	16 / 2 / 1
Fusion dim d_{fuse}	8
History length L	3
COMA samples per update	30
Mask-embedding init	57.0 (median nonzero demand)
Learning rate	1×10^{-4}
Epochs (warmup / early stop)	60 (4 / on)
Restart screening	3 inits
Batch size	20
Untrained control	0 epochs, 1 init
Repair budget (deployed)	2 sweeps-of-ten
Trainable parameters	4,468

tests; we report win counts and medians rather than pooled means. Throughout, the *training gain* of a configuration is the median over seeds of the per-seed relative difference $(\text{MLU}_{\text{untrained}} - \text{MLU}_{\text{trained}}) / \text{MLU}_{\text{untrained}}$, so positive values mean training helps. This paired statistic and the difference of the two median MLUs need not agree — not even in sign — when seed-level rankings disagree; we therefore treat the paired statistic as authoritative and report medians only for reference.

Implementation. Table 5 lists the hyperparameters, drawn from the code defaults and held fixed across all ρ and seeds. The untrained control shares the trained model’s random initialization and differs only in skipping gradient updates (`epochs=0`); it is therefore a clean ablation of the *training* factor with identical architecture, mask embedding, and repair layer.

5.2 RQ1: Does Learning Contribute When Repair Is Present?

No. Figure 4 compares the trained system against the untrained control across all five observability levels, both with the default two-sweep repair, and table 6 gives the paired statistics per ρ . The trained system wins only 9 of 25 paired cells, with a median paired gain of -1.8% — marginally in favour of the untrained control — and a sign test does not reject indifference ($p = 0.115$ in the untrained direction; the trained direction is far from significance).

A non-significant test alone cannot establish the absence of an effect, so we bound the effect size directly. Across the 25 paired cells the mean paired gain is -2.49% with a standard deviation of 7.38 points, giving a 95% confidence interval of $[-5.54\%, +0.56\%]$. The upper end is the informative one: we can *rule out* any training-induced improvement larger than 0.6%. This is a positive statement of absence rather than a failure to detect — whatever the network learns, repair leaves at most half a percent of it visible at the output. The pipeline output is essentially determined before the learned weights are consulted.

The per- ρ breakdown adds a structured detail we report for completeness: the sign of the effect is not uniform. At the sparsest setting ($\rho = 0.02$) training is mildly beneficial

TABLE 6

RQ1 per- ρ breakdown with repair enabled (5 seeds each). Trained/Untrained columns are median MLU; Gain is the median paired training gain (section 5.1), positive meaning training helps. The effect straddles zero, and the overall 95% interval $[-5.54\%, +0.56\%]$ rules out improvements beyond 0.6%.

ρ	Trained	Untrained	Gain	Wins
0.02	1.665	1.600	+4.8%	3/5
0.05	1.625	1.634	+0.0%	2/5
0.1	1.763	1.564	-1.3%	2/5
0.3	1.614	1.568	-5.4%	1/5
0.5	1.587	1.543	-3.3%	1/5
all	—	—	-1.8%	9/25

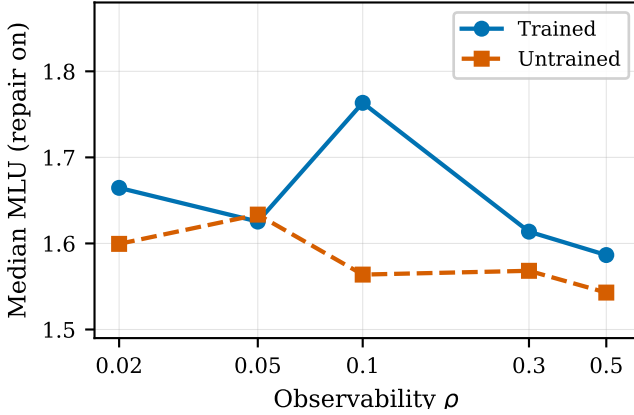


Fig. 4. Trained vs. untrained SITE with repair enabled. Training yields no measurable gain at any observability level.

(+4.8%, 3/5 wins), whereas from $\rho = 0.1$ upward the untrained control is consistently ahead, taking 3/5, 4/5, and 4/5 cells as ρ grows. We do not claim that training is *harmful* — the interval above comfortably includes zero — but the absence of any regime in which repair-backed training clearly helps is itself the finding.

Repair acts as a switch, not a dial. To establish that repair is what erases the learned signal — rather than merely coinciding with its absence — we sweep the repair budget from zero upward with everything else held fixed, at $\rho=0.3$ — a representative mid-range observability level and the one reused throughout the architecture analyses of section 5.7. Figure 5 and table 7 report both the absolute medians and the paired training gain at each budget, and the two tell opposite stories. In *absolute* terms repair is enormously valuable: it lowers the median MLU from 2.83 to 1.60, a 44% improvement that no amount of training comes close to — though section 5.9 will show that part of this figure is the information repair reads rather than the rebalancing itself. In *differential* terms it is destructive: the training gain collapses from +28.1% (3/3 paired wins) at zero budget to -3.3% (1/3) after a single unit, and never recovers at two or five, whose medians differ by less than the seed spread with no monotone trend. The transition completes within the first unit — there is no intermediate budget at which learning and repair each contribute a share.

Why should one sweep suffice? The rebalancing ranks each demand's paths by the bottleneck utilization of the

TABLE 7

Repair acts as a switch, not a dial: median MLU and paired training gain at $\rho = 0.3$ as the repair budget grows (3 seeds throughout, so columns are directly comparable; with 5 seeds the 2-sweep medians are 1.568/1.614 with gain -5.4%, 1/5 wins — same conclusion). Repair improves absolute MLU by 44% while erasing the entire training differential within a single budget unit.

	repair budget (sweeps of ten)			
	0	1	2	5
Untrained (median MLU)	2.833	1.597	1.599	1.599
Trained (median MLU)	2.184	1.655	1.649	1.649
Training gain	+28.1%	-3.3%	-9.3%	-6.3%
Paired wins	3/3	1/3	1/3	1/3

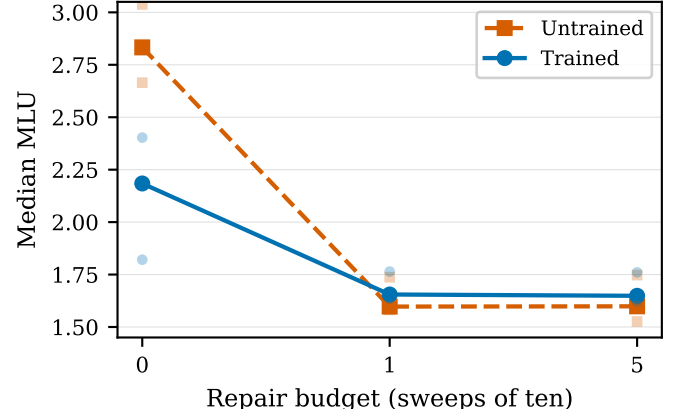


Fig. 5. Repair acts as a switch: median MLU at $\rho = 0.3$ as the repair budget grows (3 seeds; the 2-sweep plateau sits between the two rightmost points and is reported in table 7). A single sweep collapses both curves onto the same plateau, erasing the +28% training gain visible at budget zero.

links they traverse and shifts flow down that ranking (section 4.6). The ranking is dominated by where the traffic actually is, not by the split ratios it started from, so a handful of sweeps drives very different inputs — learned or random — towards the same local equilibrium, and the differences the network learned are rebalanced away. Repair thus *absorbs* whatever the network has learned, and the two components behave as functional substitutes rather than complements.

There is a second, sharper reason, and it is the one section 5.9 pursues: to rank paths by utilization at all, the repair step must know demand volumes. In the configuration measured above — the one a Teal-style system inherits — it reads them from the *ground-truth* matrix. Repair therefore enters the comparison with strictly more information than the policy, which sees only a fraction ρ of the same matrix. Under that asymmetry, it is not surprising that learned imputation adds nothing: repair already knows what the imputation was trying to guess.

5.3 RQ2: When Does Learning Contribute Without Repair?

Removing the repair layer exposes the learned component's true contribution (fig. 6). Training helps at *every* observability level we tested, winning 13 of 15 paired cells across

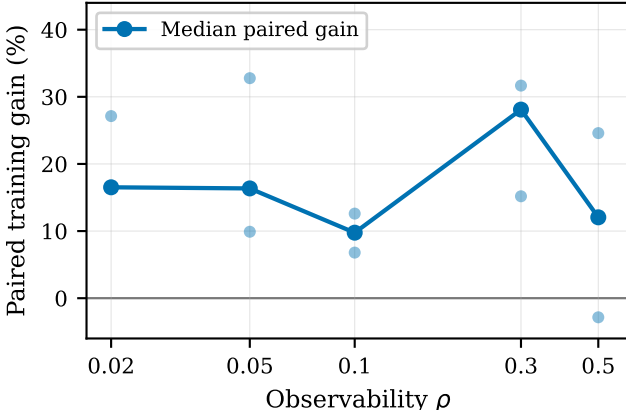


Fig. 6. Training gain without repair as a function of observability. Learning improves over the untrained control at every ρ tested; the gain is non-monotone but never collapses.

$\rho \in \{0.02, 0.05, 0.1, 0.3, 0.5\}$ (two-sided sign test $p = 0.018$). The gain is non-monotone but never collapses: it peaks at +28.1% (median) at $\rho = 0.3$, dips to +9.8% at $\rho = 0.1$, and rebounds to +16.4% ($\rho = 0.05$) and +16.5% ($\rho = 0.02$). Even at the sparsest setting we measure, two of three seeds still improve over the untrained control.

The absence of a collapse boundary is itself the finding. We expected, from the gradient-diagnosis in section 4.2, that extreme sparsity would starve the learned signal; instead the mask embeddings continue to regularize allocations even when only 2% of demands are observed, and the remaining 98% fill values do not destabilize the softmax over path logits. What does vary is the gain shape: the midband peak at $\rho = 0.3$ reflects the regime where observations are informative enough to steer allocations but not dense enough to make learning redundant, while the rebound at $\rho \leq 0.05$ reflects seed variance in the untrained control (some random seeds produce particularly bad allocations that training corrects).

5.4 RQ3: What Does Learning Still Buy? Stability.

With repair present, the trained system's advantage over the standard zero-fill baseline is marginal in the median: 17 wins in 25 paired cells (one-sided $p = 0.054$, just shy of significance). The robust advantage lies in the *tails*. Table 8 and fig. 7 report the seed spread and the worst-case MLU per ρ : the trained system has the smaller spread at all five observability levels and the better worst case at four of five, with the margin widening as sparsity grows — +38.0% at $\rho = 0.02$, where zero-filling's worst seed degrades to 2.734 while the trained system stays at 1.694. Learned mask embeddings act as a calibrated prior that protects against unlucky demand-seed combinations, which is precisely the property a controller needs under telemetry degradation.

A subtler competitor is the neighbor-aware fill (*nbr*, section 4.2): overall the trained system wins only 12/25 paired cells, but the wins and losses are *structured* — SiTE dominates at $\rho \geq 0.3$ (4/5, 3/5) and loses at $\rho \leq 0.1$. Local interpolation is a strong prior when observations are few and spatially clustered; learned embeddings earn their keep exactly when enough observations exist to inform them.

TABLE 8

Stability under repair: seed spread (max-min) and worst-case MLU of SiTE vs. zero-fill (5 seeds). Spread: SiTE wins 5/5; worst case: 4/5.

ρ	Spread		Worst case	
	SiTE	zero-fill	SiTE	zero-fill
0.5	0.358	0.456	1.940	1.913
0.3	0.164	0.472	1.761	2.066
0.1	0.398	0.466	1.917	2.072
0.05	0.306	0.432	1.781	2.072
0.02	0.269	1.248	1.694	2.734

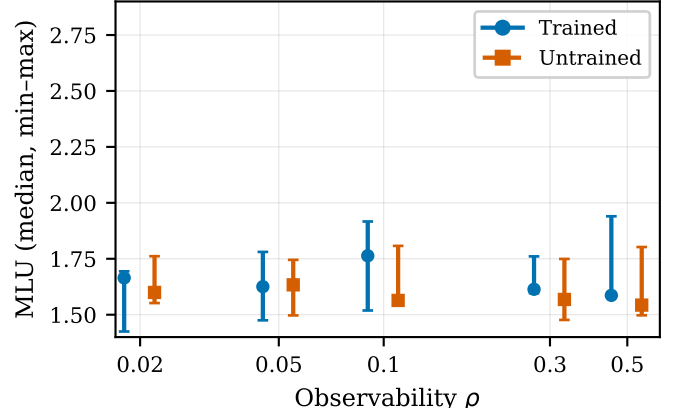


Fig. 7. Where learning still pays: per- ρ MLU with repair on (markers = median, whiskers = min-max over 5 seeds). The trained system's whiskers are tighter at every ρ and its worst case is markedly better as sparsity grows — the stability advantage that survives repair.

This crossover refines the window of section 5.3: it is where learning beats not only no-learning but also the best hand-crafted prior.

5.5 The Full-Observation Paradox

A sanity check exposes how completely repair dominates the pipeline. Information ordering demands that more observation never hurts, yet the trained system at $\rho \leq 0.5$ *beats* the same model given the complete traffic matrix at four of five ρ values (median MLU 1.5865–1.6646 vs. 1.6918 at $\rho = 1$). The paradox dissolves once repair enters the picture: repair flattens the performance landscape to the point where seed noise exceeds the information effect, and sparse models win some seeds by luck. End-to-end metrics under repair therefore cannot certify information value — or learning value — at all.

This invites a fair objection: if repair flattens the landscape until seed noise dominates, can any claim in this section be trusted? The answer is that noise domination and our two positive claims are the same observation seen from different sides. RQ1 asserts an *absence* of effect under repair, which is exactly what a noise-dominated landscape produces — and the confidence interval of section 5.2 quantifies how little room is left for one. RQ3 asserts an advantage in *spread and worst case*, quantities defined over the seed ensemble rather than contaminated by it. What noise domination does invalidate is any claim of a median improvement under repair — which is precisely the claim

TABLE 9

Cross-scale mechanism check on the 528-satellite instance (median MLU and paired gain over 3 seeds). Absorption reproduces: with repair the gain straddles zero (3/6 wins overall). Without repair the learning peak sits at $\rho=0.05$ rather than at $\rho=0.3$ as on the main topology. At repair-off $\rho=0.3$ the paired gain and the difference of medians disagree in sign; the paired value governs (section 5.1).

Repair	ρ	Trained	Untrained	Gain	Wins
on	0.3	2.91	2.97	+5.7%	2/3
	0.05	2.86	2.89	-0.6%	1/3
off	0.3	3.69	3.83	-3.2%	1/3
	0.05	3.04	4.02	+24.4%	3/3

we decline to make, and precisely the claim that the conventional end-to-end protocol would have licensed.

5.6 Cross-Scale Replication

To test whether absorption depends on constellation scale, we replicate the decisive cells on a one-third-scale instance of the same Shell-1 architecture (528 satellites, 2,112 ISLs, 93 traffic snapshots, demand density $\sim 10\times$ higher per node pair). Table 9 and fig. 8 report the trained/untrained pairing with and without repair at $\rho \in \{0.3, 0.05\}$.

Absorption replicates. With repair enabled, training again fails to establish a consistent direction: it takes 3 of 6 paired cells, with a gain of +5.7% at $\rho = 0.3$ (2/3 wins) and -0.6% at $\rho = 0.05$ (1/3) — straddling zero exactly as on the main constellation, with absolute medians differing by at most 0.06. Repair saturates and absorbs the learned signal at this scale too.

The window shifts, not vanishes. Without repair, the picture diverges from the main topology. On the 1,584-satellite constellation, training peaked at $\rho = 0.3$ (+28.1%) and stayed positive at $\rho = 0.05$ (+16.4%). On the 528-node instance the peak moves to $\rho = 0.05$ (+24.4%, 3/3 paired wins), while $\rho = 0.3$ becomes inconsistent (-3.2% paired median, 1/3 wins). This last cell is worth a word of caution: its difference-of-medians has the *opposite* sign (3.69 vs 3.83), because two seeds rank one way and the third, with a large margin, the other; following the convention of section 5.1 we report the paired value and treat the cell as unresolved rather than as a win. The learning signal is still present — it is just repositioned by the heavier per-node load. This is the stronger form of the core finding: absorption is *global* (invariant across scale), but the no-repair learning boundary is *local* (it shifts with traffic density).

5.7 Does Absorption Reproduce on a Different Architecture?

A natural worry is that absorption is a quirk of our particular model rather than a property of the repair layer. We test this by re-running the paired protocol on the closest prior sparse-input TE architecture, TEST [14]: zero-filling for unobserved demands, a Transformer over the observation history, and an ungated GCN over the topology (`-mask-mode zero -no-gate -hist-len 3`). This is the combination our ablations deliberately omit — zero-fill plus Transformer plus no mask-aware gate — and it isolates the mask-aware design as the only structural difference from SiTE.

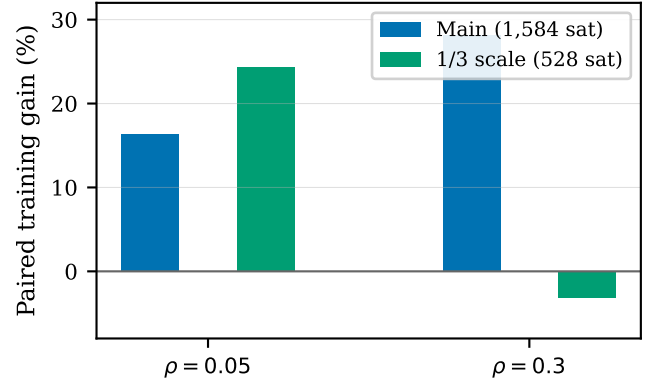


Fig. 8. The learning peak shifts with scale, not vanishes. Without repair, the paired training gain on the 528-satellite instance peaks at $\rho = 0.05$ (+24.4%), whereas on the main 1,584-satellite constellation it peaks at $\rho = 0.3$ (+28.1%). Absorption is global; the no-repair boundary is local.

TABLE 10

Does absorption reproduce on a different architecture? TEST-style baseline (zero-fill + Transformer + ungated GCN, [14]) under the same protocol: median MLU and paired gain over 3 seeds. Unlike SiTE (section 5.2), training retains value under repair.

ρ	Repair	Trained	Untrained	Gain	Wins
0.3	on	1.57	1.71	+8%	2/3
0.3	off	2.14	2.59	+17%	3/3
0.05	on	1.57	1.58	+2%	2/3
0.05	off	2.14	2.55	+14%	3/3

Table 10 reports the four decisive cells. The contrast with section 5.2 is stark. Under repair at $\rho = 0.3$, where SiTE is fully absorbed (trained $\approx$ untrained, 9/25 paired wins), the TEST-style architecture is *not* absorbed: training wins 2/3 seeds with a +8% median gain. Without repair the two architectures agree — training helps strongly (+17%, 3/3) — so the divergence is specifically in how repair interacts with the learned policy, not in whether learning helps at all.

Absorption is architecture-specific. On a mask-unaware architecture the untrained control is unstable — seed spread at $\rho = 0.3$ under repair spans 1.56 to 1.89, against the trained control's tight 1.57–1.57 — so training retains value as a stabiliser even when repair is on. On our mask-aware model the untrained control is already competent (a random gate produces trivial imputation that oracle repair handles well), so training and untraining converge under repair. Absorption is therefore not a universal property of constraint-repair layers; it arises where the learned component and repair solve the same imputation subproblem, so repair subsumes the policy.

The gate drives absorption. Table 11 isolates the component. Holding the fill strategy fixed, turning the gate on flips a non-absorbing cell (2/3 wins, +2% to +8%) into an absorbing one (0/3, -2% to -14%); holding the gate fixed, swapping the learnable placeholder for zero-fill changes the magnitude but never the verdict. The mask-aware gate is what creates the overlap with repair: with the gate, the policy's routing is observation-conditioned, so oracle repair — which also conditions on the true demands — subsumes

TABLE 11

Which component drives absorption? A 2×2 ablation at $\rho = 0.3$ under oracle repair (3 seeds, paired wins / median gain). The mask-aware gate, not the learnable placeholder, determines whether absorption occurs: every gate-on cell absorbs, every gate-off cell does not.

Fill strategy	gate on	gate off
embed (placeholder)	0/3, -2%	2/3, +2%
zero (no placeholder)	0/3, -14%	2/3, +8%

it; without the gate, the policy routes mask-agnostically and retains value repair cannot replicate. Absorption is thus a property of mask-aware routing under oracle repair, not of the repair layer or the placeholder alone.

5.8 Inference Latency

Although latency is not the subject of this study, it determines whether the mechanism we analyse is relevant to a deployable controller. On the 1,584-satellite constellation, one full inference pass — demand-set assembly, graph rebuild, gated GNN, temporal encoder, fusion, policy head, and the two-sweep repair — completes in under 0.4 s per snapshot on a single GPU, against a control-loop period of 5 minutes. Repair is structurally cheap — two sweeps amount to twenty sparse scatter operations over the path-link incidence — which is precisely why it is attractive to keep, and why its interaction with learning matters: inexpensive in latency, but, as section 5.2 shows, expensive in learned signal.

5.9 Repair Without the Oracle

Every result so far granted the repair step the ground-truth traffic matrix, because that is what the inherited implementation does (section 4.6). Under sparse observation this is not a reproducible deployment: a controller metering ρ of its demands cannot compute true link utilization. We therefore re-run the decisive cells with repair restricted to what the controller can actually see — OBS-ZERO and OBS-NBR — while the realized MLU is still scored against the true demands, as everywhere else.

Repair’s advantage is largely informational. Table 12 places the four repair settings on a common scale using the untrained control at $\rho = 0.3$, so that no learning is involved and only the information available to repair varies. The ordering is unambiguous: repair remains worth having when fed the observation — neighbour-filled repair still cuts median MLU from 2.83 to 1.74 — but it is markedly weaker than the oracle version at 1.60, and zero-filled repair is weaker still at 2.30. A substantial part of what looked like the power of *repair* in section 5.2 is in fact the power of the *ground-truth matrix* that repair was quietly reading.

Does learning matter again? Removing the oracle does not cleanly restore learning’s value. Under OBS-NBR repair, the trained policy wins 2/3 seeds at $\rho=0.1$ (+1% median gain) and $\rho=0.5$ (+6%), yet loses at $\rho=0.3$ (−8%, 0/3) and $\rho=0.05$ (−9%, 1/3). Under OBS-ZERO repair, training wins decisively at $\rho=0.05$ (+18%, 3/3) but is flat at $\rho=0.3$ (−1%, 1/3). The absorption of section 5.2 is therefore *partly* an oracle artifact—at $\rho=0.1$ and 0.5 learning helps once repair

TABLE 12

What repair is worth once it can only read the observation: median MLU of the *untrained* control at $\rho = 0.3$ (3 seeds), so that only the information given to repair varies.

Repair setting	Matrix repair reads	Median MLU
none	—	2.833
OBS-ZERO	observation, zeros elsewhere	2.295
OBS-NBR	observation, neighbour fill	1.740
ORACLE	ground truth	1.599

reads only the observation—but it is not fully explained by the oracle: at $\rho=0.3$, the densest point we test, training still does not help even when repair is denied the ground truth. The contrast at $\rho=0.05$ sharpens the point: under weak OBS-ZERO repair training helps (+18%), yet under the stronger OBS-NBR repair it does not (−9%) — so it is the repair step’s structural rebalancing, not the oracle alone, that absorbs the policy’s contribution where repair is strong.

5.10 Takeaways

- **Evaluation.** A learned TE system paired with a constraint-repair layer must be benchmarked repair-free or against repair-only controls; end-to-end numbers under repair conflate the two components and can certify a model whose weights contribute nothing. Report, in addition, *which* traffic matrix the repair step is allowed to read: a repair layer holding the ground truth is not the same system as one holding the observation (section 5.9).
- **Design.** If repair is kept for safety, spend the learning budget on what repair cannot provide — tail robustness (section 5.4) — and evaluate it on spread and worst case, not medians.
- **Deployment.** Learning is effective across the entire observability range once repair is removed (13/15 paired wins), so there is no lower observability cutoff below which training should be abandoned. The operative choice is not whether to pay for repair but which matrix to hand it: the oracle-fed variant that erases the learned differential is precisely the variant a sparsely-measuring controller cannot run.
- **Scope of sections 5.2 and 5.4.** Those results characterize the oracle-fed configuration, which is what current systems inherit and report. The deployable variant (section 5.9) softens but does not erase absorption — a residual remains at high ρ — so the oracle measurements are best read as an upper bound on absorption, not a deployable forecast. Conflating the two readings is how the effect stayed invisible.

6 CONCLUSION

This paper presented a controlled mechanism study of learned traffic engineering under sparse observations, using SiTE as the vehicle. The central finding is *repair absorption*: when a constraint-repair layer is present, it acts as a switch rather than a dial — a single rebalancing sweep collapses the entire training differential, and further sweeps change nothing — leaving a trained network indistinguishable from

random initialization (9/25 paired wins on a 1,584-satellite Starlink constellation, with a 95% interval on the paired gain that rules out any improvement beyond 0.6%; 3/6 on a one-third-scale instance). The absorption is global across observability levels, random seeds, and constellation scales, but architecture-specific: it does not reproduce on a mask-unaware baseline (section 5.7), which refines its scope to the mask-aware design rather than the repair layer alone.

Without repair, learning is effective across the entire observability range tested (13/15 paired wins, $\rho \in [0.02, 0.5]$), but the gain profile is topology-dependent: the learning peak sits at $\rho = 0.3$ on the main constellation and shifts to $\rho = 0.05$ on the smaller, denser instance. The one robust residual advantage of the trained system is stability — lower seed spread and better worst-case MLU (up to +38% at extreme sparsity) — not median performance.

The mechanism has a cause that the prevailing evaluation protocol keeps out of view. To rank congested paths, repair needs demand volumes, and the implementation inherited from fully-observed WAN settings takes them from the ground-truth traffic matrix. Repair therefore competes with the learned imputation while already knowing what that imputation was trying to estimate — an asymmetry that is unremarkable when the matrix is known, and unavailable when it is not. Restricting repair to the sparse observation costs it a substantial part of its advantage, which separates two things the literature reports as one: the repair *algorithm* and the repair *information*.

This distinction also fixes the scope of our results. The absorption is largely an artifact of the oracle-fed configuration current systems inherit and report: the oracle variant systematically overstates it, and a residual absorption persists even when repair reads only the observation at high ρ (section 5.9), so the effect is not purely informational. The prevailing evaluation protocol amplifies it, and conflating the protocol diagnosis with a deployable forecast is precisely how the effect stayed invisible.

The guidance follows directly. For evaluation, learned TE systems paired with repair must be benchmarked repair-free or against repair-only controls, and must state which matrix repair reads. For design, if repair is retained for safety, the learning budget should target tail robustness rather than median improvement. For deployment, training remains worthwhile even at 2% observability, and the operative question is not whether to pay for repair but which matrix to hand it.

Three directions remain open. First, whether absorption survives when the heuristic rebalancing is replaced by a genuine differentiable projection — dual-based projection or a feasibility layer in differentiable optimization — and whether such a layer, restricted to the observation, absorbs less. Second, the stability advantage suggests a dedicated training objective, explicitly optimizing worst-case MLU rather than the mean, may unlock what the current loss leaves on the table. Third, measurement scheduling — deciding *which* demands to observe next — becomes especially valuable once repair no longer has free access to the full matrix, since each observation then improves the allocator and the repair step at once.

REFERENCES

- [1] M. Handley, “Delay is not an option: Low latency routing in space,” in *Proceedings of the 17th ACM Workshop on Hot Topics in Networks (HotNets)*, 2018.
- [2] D. Bhattacharjee and A. Singla, “Network topology design at 27,000 km/hour,” in *Proceedings of the 15th International Conference on Emerging Networking Experiments and Technologies (CoNEXT)*, 2019.
- [3] G. Giuliari, T. Klenze, M. Legner, D. Basin, A. Perrig, and A. Singla, “Internet backbones in space,” *ACM SIGCOMM Computer Communication Review*, vol. 50, no. 1, 2020.
- [4] S. Kassing, D. Bhattacharjee, A. B. Águas, J. E. Saethre, and A. Singla, “Exploring the “internet from space” with hypatia,” in *Proceedings of the ACM Internet Measurement Conference (IMC)*, 2020.
- [5] F. Michel, M. Trevisan, D. Giordano, and O. Bonaventure, “A first look at starlink performance,” in *Proceedings of the ACM Internet Measurement Conference (IMC)*, 2022.
- [6] S. Jain *et al.*, “B4: Experience with a globally-deployed software defined WAN,” in *Proceedings of the ACM SIGCOMM 2013 Conference*, 2013, tODO: full author list.
- [7] C.-Y. Hong, S. Kandula, R. Mahajan, M. Zhang, V. Gill, M. Nanduri, and R. Wattenhofer, “Achieving high utilization with software-driven WAN,” in *Proceedings of the ACM SIGCOMM 2013 Conference*, 2013.
- [8] C.-Y. Hong *et al.*, “B4 and after: Managing hierarchy, partitioning, and asymmetry for availability and scale in google’s software-defined WAN,” in *Proceedings of the ACM SIGCOMM 2018 Conference*, 2018, tODO: full author list.
- [9] H. Wu, Y. Han, M. Rajpal, Q. Zhang, and J. Wang, “SaTE: Low-latency traffic engineering for satellite networks,” in *Proceedings of the ACM SIGCOMM 2025 Conference*, 2025, pp. 896–916.
- [10] Z. Xu, F. Y. Yan, R. Singh, J. T. Chiu, A. M. Rush, and M. Yu, “Teal: Learning-accelerated optimization of WAN traffic engineering,” in *Proceedings of the ACM SIGCOMM 2023 Conference*, 2023, pp. 378–393.
- [11] Y. Perry, F. V. Frujeri, C. Hoch, S. Kandula, I. Menache, M. Schapira, and A. Tamar, “DOTE: Rethinking (predictive) WAN traffic engineering,” in *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, 2023, pp. 1557–1581.
- [12] A. A. AlQiam, Y. Yao, Z. Wang, S. A. Ahuja, Y. Zhang, S. G. Rao, B. Ribeiro, and M. Tawarmalani, “Transferable neural WAN TE for changing topologies,” in *Proceedings of the ACM SIGCOMM 2024 Conference*, 2024, pp. 86–102.
- [13] S. Boyd, N. Parikh, E. Chu, B. Peleato, and J. Eckstein, “Distributed optimization and statistical learning via the alternating direction method of multipliers,” *Foundations and Trends in Machine Learning*, vol. 3, no. 1, 2011.
- [14] Y. Guo, Z. Huang, M. Ding, and H. Luo, “An end-to-end learning approach for traffic engineering with sparse traffic measurements,” *IEEE Transactions on Mobile Computing*, vol. 25, no. 4, pp. 5448–5463, 2026, tODO: verify author list against the published version.
- [15] B. Fortz and M. Thorup, “Internet traffic engineering by optimizing OSPF weights,” in *Proceedings of IEEE INFOCOM 2000*, 2000.
- [16] H. Wang, H. Xie, L. Qiu, Y. R. Yang, Y. Zhang, and A. Greenberg, “COPE: Traffic engineering in dynamic networks,” in *Proceedings of the ACM SIGCOMM 2006 Conference*, 2006.
- [17] P. Kumar, Y. Yuan, C. Yu, N. Foster, R. Kleinberg, P. Lapukhov, C. L. Lim, and R. Soulé, “Semi-oblivious traffic engineering: The road not taken,” in *15th USENIX Symposium on Networked Systems Design and Implementation (NSDI 18)*, 2018, pp. 157–170.
- [18] F. Abuzaid, S. Kandula, B. Arzani, I. Menache, P. Bailis, and M. Zaharia, “Contracting wide-area network topologies to solve flow problems quickly,” in *18th USENIX Symposium on Networked Systems Design and Implementation (NSDI 21)*, 2021, tODO: verify author list.
- [19] D. Narayanan, F. Kazhamiaka, F. Abuzaid, P. Kraft, A. Agrawal, S. Kandula, S. Boyd, and M. Zaharia, “Solving large-scale granular resource allocation problems efficiently with POP,” in *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles (SOSP)*, 2021, tODO: verify author list.
- [20] A. Valadarsky, M. Schapira, D. Shahaf, and A. Tamar, “Learning to route,” in *Proceedings of the 16th ACM Workshop on Hot Topics in Networks (HotNets)*, 2017.

- [21] X. Liu *et al.*, "FIGRET: Fine-grained robustness-enhanced traffic engineering," in *Proceedings of the ACM SIGCOMM 2024 Conference*, 2024, tODO: verify author list and pages.
- [22] D. Wu *et al.*, "NetLLM: Adapting large language models for networking," in *Proceedings of the ACM SIGCOMM 2024 Conference*, 2024, tODO: verify author list and pages.
- [23] Y. Lu, Y. Zhao, F. Sun, and H. Li, "Virtual topology for LEO satellite networks based on earth-fixed footprint mode," *IEEE Communications Letters*, vol. 17, no. 2, pp. 357–360, 2013, tODO: verify author list.
- [24] M. Hu, M. Xiao, W. Xu, T. Deng, Y. Dong, and K. Peng, "Traffic engineering for software-defined LEO constellations," *IEEE Transactions on Network and Service Management*, vol. 19, no. 4, pp. 5090–5103, 2022, tODO: verify author name spellings.
- [25] L. Wei, T.-T. Le, Y. Ji, M. Wang, Y. Liu, Y. Wang, and J. C. Lui, "Towards efficient traffic engineering via distributed optimization in large-scale LEO constellation," in *2024 IEEE International Conference on Communications Workshops (ICC Workshops)*, 2024, pp. 353–358, tODO: verify co-author name spellings.
- [26] X. Liu, Z. Zhang, X. Qin, H. Zhou, and L. Zhao, "FlexSATE: Flexible and distributed traffic engineering with supervised learning in ultra-dense low-earth-orbit satellite networks," in *GLOBECOM 2024 - 2024 IEEE Global Communications Conference*, 2024, pp. 4466–4471, tODO: verify author name spellings.
- [27] Z. Yang, G. Fan, A. Cao, Y. Guo, W. Li, C. Ying, T. Liu, S. Yue, and H. Jin, "Scalable traffic allocation in dynamic networks via end-to-end imitation learning," *IEEE/ACM Transactions on Networking*, vol. 34, 2026.
- [28] Z. Yang, G. Fan, A. Cao, Y. Guo, W. Li, T. Liu, and H. Jin, "Learning to accelerate traffic allocation over large-scale networks," in *IEEE INFOCOM 2025 - IEEE Conference on Computer Communications*, 2025.
- [29] Y. Vardi, "Network tomography: Estimating source-destination traffic intensities from link data," *Journal of the American Statistical Association*, vol. 91, no. 433, 1996.
- [30] Y. Zhang, M. Roughan, N. Duffield, and A. Greenberg, "Fast accurate computation of large-scale IP traffic matrices from link loads," in *Proceedings of ACM SIGMETRICS 2003*, 2003.
- [31] Y. Zhang, M. Roughan, W. Willinger, and L. Qiu, "Spatio-temporal compressive sensing and internet traffic matrices," in *Proceedings of the ACM SIGCOMM 2009 Conference*, 2009.
- [32] C. Huo, D. Jin, Y. Li, D. He, Y.-B. Yang, and L. Wu, "T2-GNN: Graph neural networks for graphs with incomplete features and structure via teacher-student distillation," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2023.
- [33] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *International Conference on Learning Representations (ICLR)*, 2017.
- [34] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [35] J. Foerster, G. Farquhar, T. Afouras, N. Nardelli, and S. Whiteson, "Counterfactual multi-agent policy gradients," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2018.
- [36] G. Giuliani, T. Ciussani, A. Perrig, and A. Singla, "ICARUS: Attacking low earth orbit satellite networks," in *Proceedings of the 2021 USENIX Annual Technical Conference (USENIX ATC)*, 2021, pp. 317–331.